

The Harness Pattern: What I Learned Running AI Agents Against Local Models

A year of production failures taught me that the interesting engineering in AI agents isn't the model — it's the deterministic code that won't let the model lie about what it did.

Julio Rodriguez Martino · August 2026

“He was the solitary and lucid spectator of a multiform, instantaneous, and almost intolerably precise world.”

— Jorge Luis Borges, *Funes the Memorious*

I named my project Funes after Borges' character who could not forget, because the whole point was memory: a personal AI assistant that remembers what you tell it and uses that knowledge in every answer. What I didn't expect was that the interesting engineering would have nothing to do with memory, or with the model, or with prompts. It would be about the **harness** — the deterministic program that wraps the model and controls everything it cannot control about itself.

Funes is a self-hosted, multi-agent AI assistant written in C++17. It runs against local models (llama.cpp on a single GPU), or cloud providers, or both. It has persistent semantic memory backed by SQLite and sqlite-vec, a tool-calling loop, agent-to-agent delegation, a cron scheduler, WhatsApp and Gmail integrations via MCP, and a daily newsletter pipeline that searches, curates, and sends an AI news digest without human intervention.

This article is about the pattern I converged on while building it — what I call the *harness pattern* — and the specific, dated, embarrassing production failures that forced each piece into existence.

What is a harness?

The AI-agent literature talks a lot about *agentic frameworks*: orchestration layers that route messages between models, manage tool calls, and handle multi-step reasoning. The harness is a narrower, more opinionated idea. It is the **deterministic code that decides whether to trust the model's output at every step**.

The model produces text and tool calls. The harness decides:

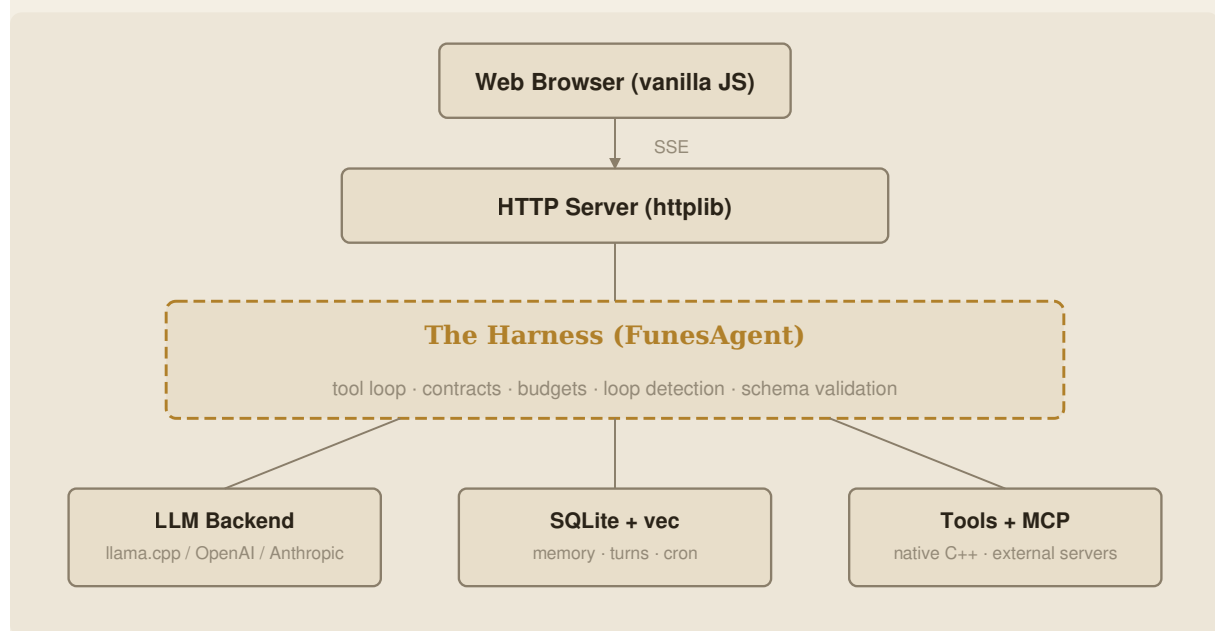
- Is this tool in the agent's allowlist?

- Has the model called this tool too many times?
- Is the model trying to finish before it has done the required work?
- Does the answer match the declared schema?
- If the run failed, does the failure say so explicitly — or does it look like content?

Think of it like a horse and rider. The LLM is the horse — powerful, but it goes where it's pointed. The harness is the rider: it sets the course, pulls the reins when needed, and decides when to stop. Without a rider, the horse eats grass.

The architecture

Funes is a single C++ binary (~10,000 lines) that serves the web UI, the REST/SSE API, the agent runtime, and the memory engine. No containers, no external databases, no package managers at runtime. Memory is one SQLite file. Agents are YAML files.



The user talks to one agent (`funes` , the orchestrator). When a request needs a specialist — deep web research, file operations, email, WhatsApp — the orchestrator *delegates* via a `delegate_to_agent` tool call. The specialist runs its own tool loop, invisible to the user, and returns its answer. The orchestrator relays the result in its own voice.

Agents are YAML: a name, a system prompt, a tool allowlist, and a handful of knobs. The shared C++ runtime executes all of them identically. A new agent is a five-line config file and a reload — no code, no rebuild, no container.

The safety mechanisms (and the failures that created them)

Every safety mechanism in Funes exists because of a specific production failure. I document them in code comments with dates. Here are the ones that hurt.

1. Completion contracts

The failure: My newsletter agent was supposed to search for news, write posts, and send an email. Instead, it described what a great newsletter it *would* send, and stopped. The model treated “narrate the plan” as equivalent to “execute the plan.” The output looked like a newsletter. It was not a newsletter.

The fix: A *completion contract* — a list of tools that must have **succeeded** before the harness accepts a text answer. If the model tries to answer before the contract is satisfied, the harness injects a nudge: “*Stop. That was a text answer, but publish_issue has not succeeded yet.*” If the model keeps dodging, the run ends in an explicit **FAILED** rather than a plausible-sounding lie.

```
# In the agent's YAML config:
require_tools: [harvest_candidates, publish_issue]
```

The contract tracks success with insert/erase semantics: a later failing call to the same tool undoes an earlier success. This matters when an agent calls the same required tool twice for different purposes — without the erase, the first success would permanently satisfy the contract regardless of what happened next.

2. Tool budgets and tool withholding

The failure (2026-07-31): My **researcher** agent issued 20 distinct **web_search** calls without ever synthesizing. Twice in one pipeline run. Half an hour of GPU time for nothing. The loop detector eventually stopped it, but stopping a run is not the same as getting an answer out of it.

The fix: Per-tool call ceilings that *refuse* the call with a readable error: “You have used all 6 **web_search** calls. Answer now from what you already have.”

The second failure (same day): The 9B model re-issued the refused **web_search** seven times in a row. Each refusal cost a step. It ran out of steps with no answer and the whole newsletter pipeline collapsed behind it.

The real fix: After a budget refusal, the harness *withholds the tool schema entirely* on the next completion. Not just **tool_choice: "none"** — the tools array is dropped from the request. The model literally cannot ask for the tool because it doesn’t know it exists anymore. And a user-turn notice tells the model that “a tool call written out as text will be discarded rather than executed” —

because yes, a local model that has seen 15 tool calls in its transcript will write one in prose if you take the API mechanism away.

3. Failure propagation

The failure: A raw web-search dump — the unprocessed output of a Tavily API call — travelled three levels up a delegation chain and was served to the user as a newsletter. The intermediate agent had no answer, so the harness used the last tool result as the response. It looked like content. It was not content.

The fix: Every failure exit now produces a message starting with `FAILED —`. The delegation tool checks for this prefix and turns it into a tool error rather than passing it up as content. The failure builders take a tool *name* and never its text, closing the path through which tool output could re-enter an answer.

The pattern: refuse recoverably, kill as backstop. Tool budget refusals are recoverable — the model is told to conclude from what it has, and usually does. The loop detector kills the run outright. Having both means the model usually produces something useful, and the backstop catches the cases where it doesn't.

Memory: the product, not the feature

Funes has persistent memory backed by SQLite and sqlite-vec. Tell it something today and it knows it tomorrow, in a different conversation. Memory is not a bolt-on retrieval-augmented generation pipeline — it is the product.

Three layers work together:

- **Automatic memory:** every exchange is stored (truncated). These are low-priority background knowledge.
- **Deliberate memory:** when you tell Funes a fact worth keeping, it calls `remember`. These get a 1.3× relevance boost over auto-memories, so taught facts outrank conversation logs at the same raw similarity.
- **Consolidation:** a background thread merges near-duplicate memories (cosine ≥ 0.92) via LLM calls and prunes stale auto-memories that have never been recalled. Deliberately taught memories are protected.

Semantic search uses an embedding model (nomic-embed-text on a local llama.cpp instance). When the embedding endpoint is down, recall degrades gracefully to keyword search. When it comes back, a

background thread backfills the missing vectors. If the embedding model changes dimension, the vector table is rebuilt automatically.

Your entire memory is `~/funes/memory.db`. Back it up with `cp`.

Multi-agent orchestration: less than you think

Funes has 12 agents. Here is the thing: they are not autonomous entities. Each one is a name, a prompt, a tool allowlist, and a handful of config knobs, run by the same shared runtime. The word “agent” is doing more marketing work than engineering work.

What makes it a real multi-agent system (if a simple one) is **delegation**. The orchestrator calls `delegate_to_agent`, which creates a new `FunesAgent` instance with the specialist’s config and runs it in-process. The specialist’s tool calls and internal reasoning are invisible to the user. Its answer comes back as a tool result.

Delegation is stateless and single-shot. The specialist sees only the task string — not the conversation, not the user’s context, not other agents’ results. If it needs to ask follow-up questions, it returns them and the orchestrator relays them to the user and re-delegates with the answers.

Safety: self-delegation is refused, depth is capped at 2, and failures are detected and surfaced as errors. The specialist never talks to the user directly.

Local-first: running on your own hardware

Funes runs entirely on a single machine I call “yoda” — a desktop with one GPU. Three llama.cpp instances serve the main LLM (port 8080), the embedding model (port 8081), and a vision model with a multimodal projector (port 8083). The C++ binary, the SQLite database, the web UI, the agent runtime, and the cron scheduler all run in the same process.

The design is deliberately provider-agnostic. The `LLMClient` speaks both the OpenAI-compatible API (llama.cpp, Groq, OpenAI) and Anthropic’s native API. Switch with one environment variable. Model-specific quirks (Qwen’s non-standard tool-result format, local models writing tool calls as JSON in prose) are handled in the client, not in the agent loop.

Running local models is where the harness earns its keep. Cloud models are well-aligned: they follow instructions, use tools through the API’s native mechanism, and stop when asked. Local models —

especially smaller ones — are creative about ignoring all of that. The harness exists because a 9B model will:

- Write a tool call as a JSON blob in prose instead of using the function-calling API
- Ignore a “stop and synthesize” instruction and make another tool call anyway
- Describe the work instead of doing it
- Search endlessly without ever producing an answer
- Re-issue a refused tool call seven times

Every one of those is a real incident. The harness catches each one.

The newsletter: deterministic over agentic

The most complex thing Funes does is produce a daily AI newsletter. The pipeline went through several iterations, and the final design embodies a principle I now apply everywhere: **only delegate to the model the steps that truly need model judgment. Everything else is code.**

STEP	WHO DOES IT	WHY
Search for news	C++ (Tavily API)	Deterministic, parameterized
Deduplicate, filter stale, cap per source	C++	Rules, not judgment
Fetch every page, drop failures	C++	I/O, not reasoning
Build numbered candidate pool	C++	Data preparation
Pick stories, write post text	LLM (curator agent)	Judgment and writing
Resolve IDs to URLs	C++	The model never touches URLs
Grounding check (word overlap)	C++	Deterministic, no LLM
Link verification, repair	Python	HTTP HEAD/GET
Render HTML, send email	Python	Template + SMTP

The model does exactly two things: decide which stories are interesting and write a sentence about each one. Everything before and after is deterministic.

The grounding check deserves a note. After the model picks stories and writes posts, `publish_issue` finds the sentence in each candidate’s page text with the highest content-word overlap against the post text. If the overlap falls below 3 words, the post is rejected — the model hallucinated a claim not supported by the source. Before rejecting, the system tries to substitute a different pool candidate whose page does support the text. All of this is deterministic string matching. No LLM involved.

The model never supplies URLs. It picks candidates by numeric ID; the harness resolves IDs to URLs from the pool file. This closes the path through which a hallucinated URL could enter a published newsletter.

MCP: tools beyond the built-in set

Funes supports the Model Context Protocol for external tool servers. Three integrations run in production: a WhatsApp Web bridge (Go + Python, vendored with a port-conflict fix), an IMAP email server (Node.js, vendored with a TLS SNI patch), and an RSS reader (npx, unpatched).

MCP tools are merged with the native C++ registry and filtered by the agent's YAML allowlist. The agent doesn't know whether a tool is native or MCP — the dispatch layer handles the difference. Native tools win on name collisions.

Two transports: SSE (HTTP) and stdio (child process). The stdio transport is how the WhatsApp and email integrations work — the harness spawns the MCP server as a child process with configured environment variables and communicates over stdin/stdout.

What I would tell myself a year ago

If I were starting over, these are the things I wish I had known:

The model is not the product. The model is a component. The product is the harness, the memory, the tools, and the integrations. When the model improves, everything gets better for free. When the model regresses, the harness catches it. Build the system around the model, not on top of it.

Every safety mechanism is a response to a specific failure. Don't design them upfront. Ship, watch what breaks, and build the smallest fix that prevents that specific failure. Completion contracts, tool budgets, loop detection, failure propagation, answer schemas — every one of these was a “*well, that went badly*” followed by a targeted intervention.

Deterministic beats agentic. If a step has one right answer, it's code, not a prompt. Three agents and 46 steps became one agent and four tool calls. The model's job got smaller, and the pipeline got more reliable.

Make failure explicit. A run that has no answer must say so in a shape a caller can detect. The alternative is raw tool output masquerading as a result, climbing a delegation chain, and ending up in someone's inbox as a newsletter that is actually a JSON dump from Tavily. Ask me how I know.

Refuse recoverably, kill as backstop. A budget refusal that says “conclude from what you have” usually produces something. A loop detector that kills the run outright usually doesn’t. Having both means the system fails gracefully most of the time and fails explicitly the rest.

Verify, don’t trust. Check the run record after the agent says it sent the newsletter. Run a grounding check after the model writes a post. Test with a mock LLM that can simulate every failure mode. The model is powerful and unreliable — the harness is limited and deterministic. Trust the harness.

Funes is open-source under the MIT license. The codebase is ~10,000 lines of C++17, ~1,600 lines of Python, and ~1,500 lines of vanilla JavaScript, with 22 unit tests and a 556-line integration test suite. It runs on a single machine with one GPU.